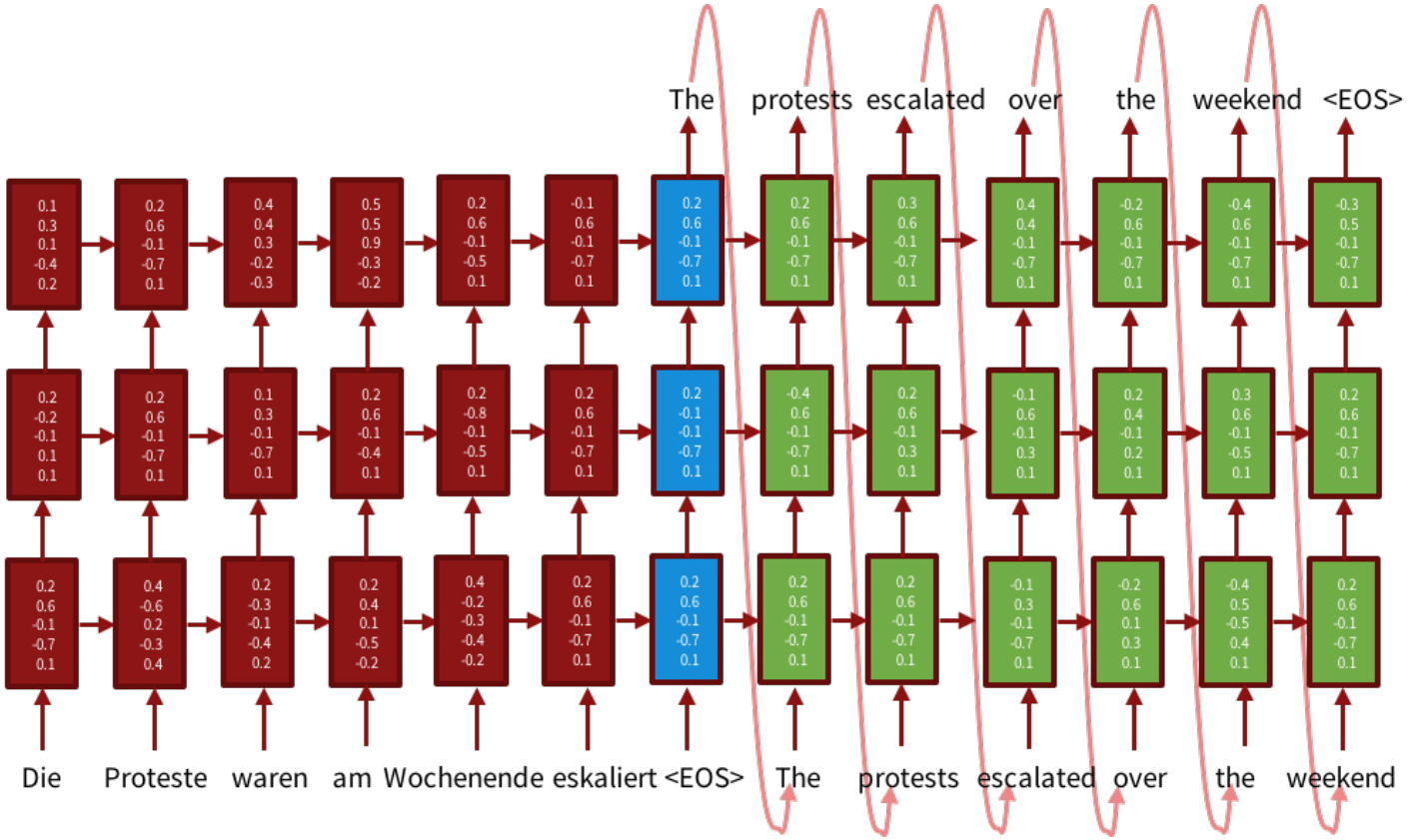
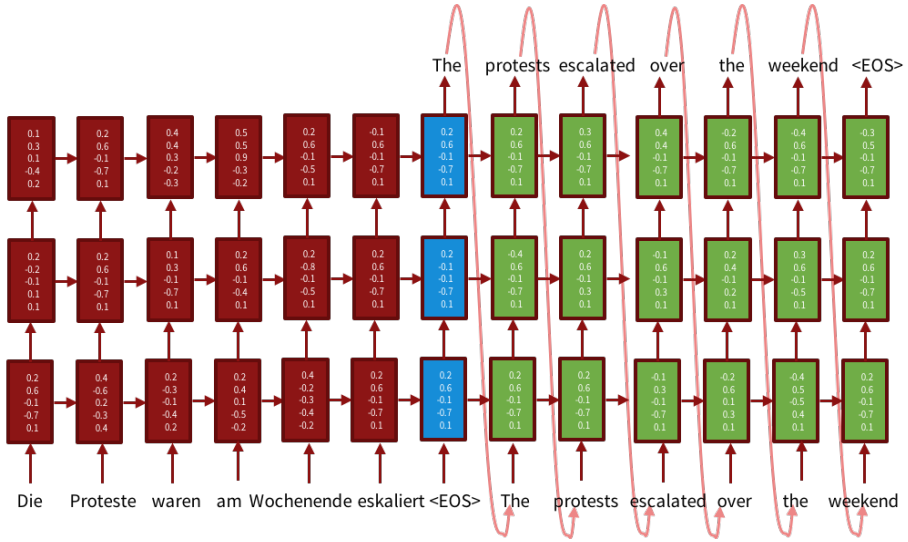
Neural Machine Translation

- The current state of the art outperforms phrase-based translation by a considerable margin in a variety of settings
- The standard architecture is called encoder-decoder

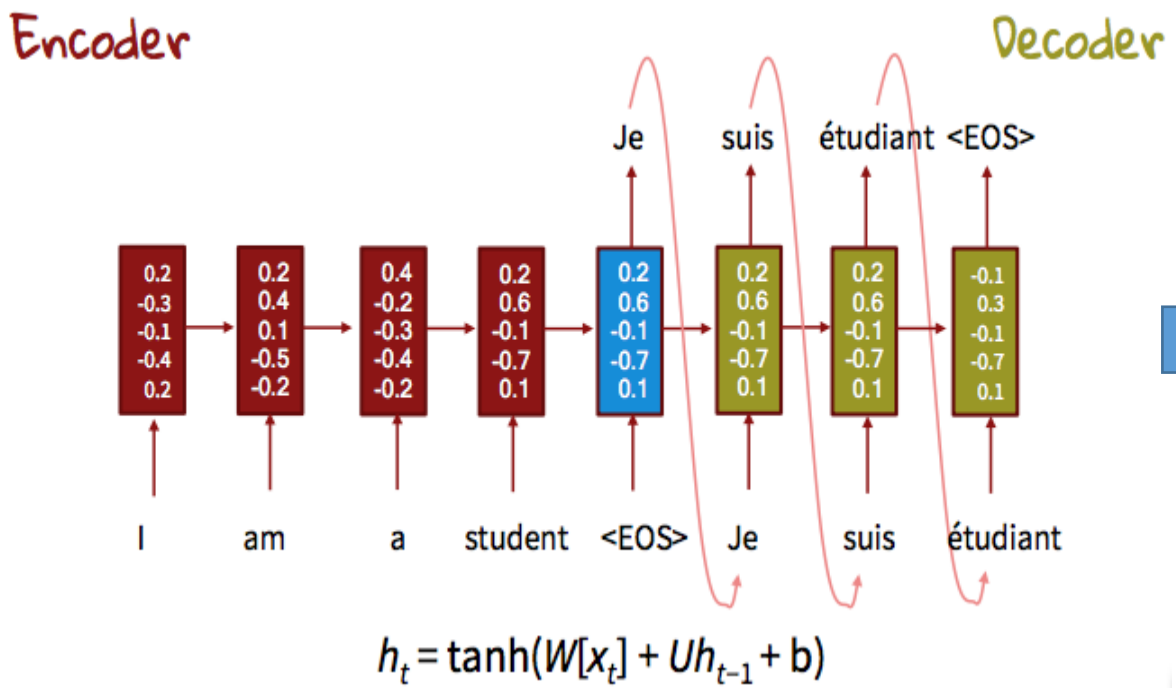


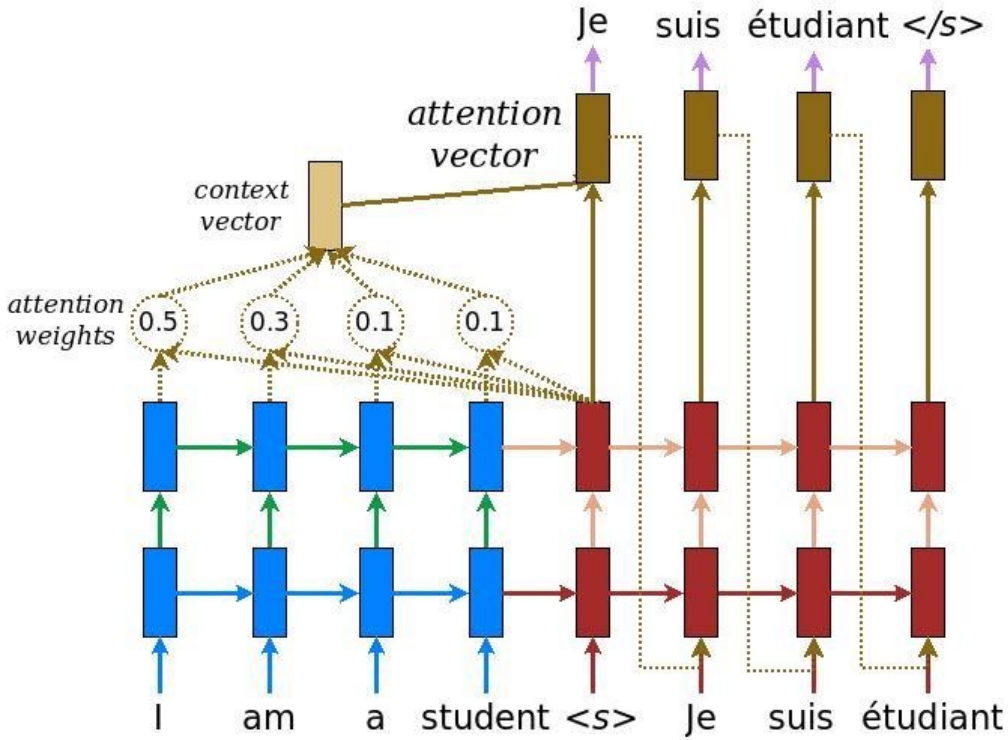
Neural Machine Translation

- Encoder-decoder systems have two stages:
  1. Encoding: the input sentence is encoded into a vector. This can be done by RNNs or other types of networks.
  2. Decoding: like a language model it assigns a probability to the next word. However, it also conditions on the encoded input.
- This is a neural LM, which is conditioned on the encoding of the input



RNN+Attention





RNN+Attention

- Attention is a mechanism for making specific parts of the input more accessible to the decoder
- Instead of the decoder only receiving the summary vector for the input (the last hidden state), using attention it receives a linear combination of the hidden states of the tokens of the sequence that are most relevant for the next decoded word
  - A form of a soft alignment
- The coefficients are themselves dynamically computed
  - We will define this formally in the next slides

Neural Machine Translation

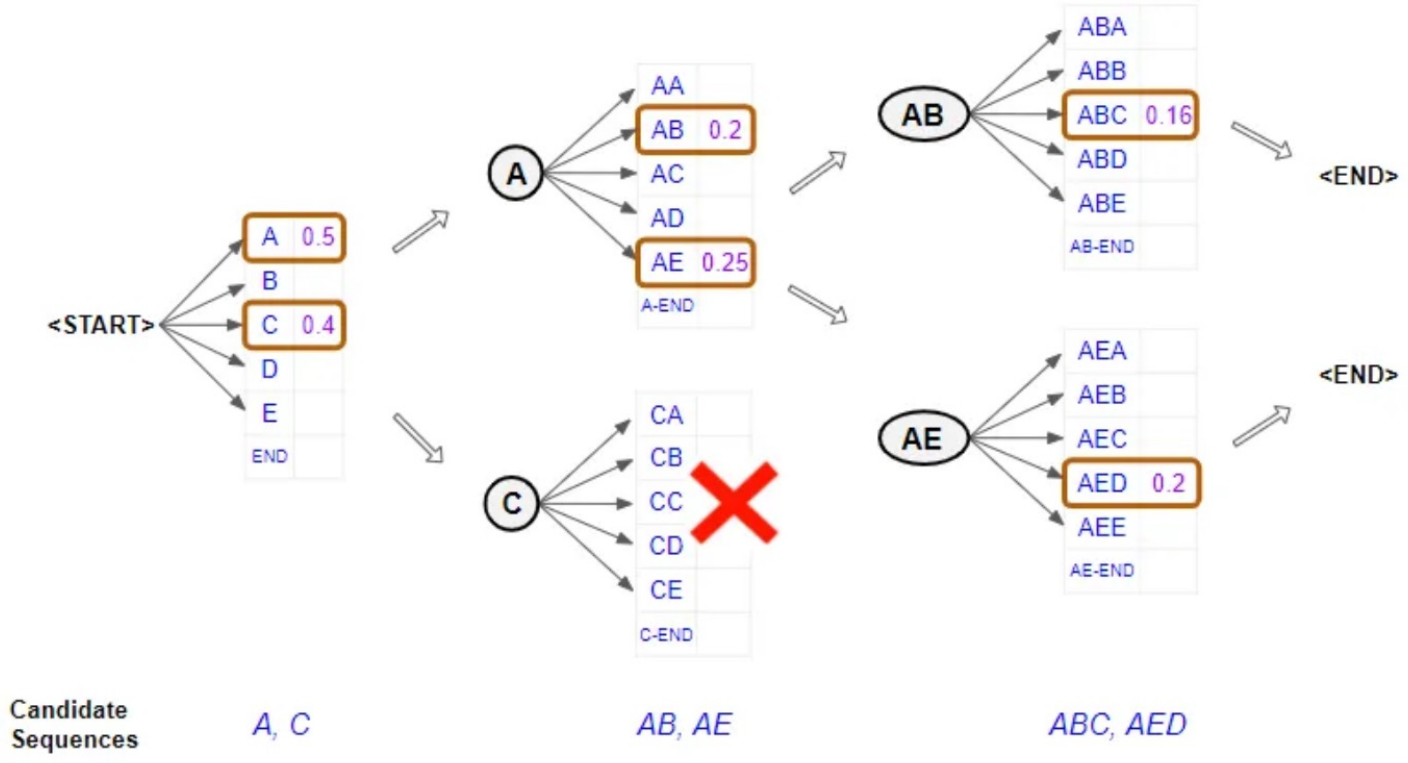
- Training: standard training is done by maximizing the average log probability of the next word, given the input and the prefix (*y* is the translation, *x* is the input sentence)

$$L(\theta) = \frac{1}{N} \sum_{i=1}^{N} log[p_\theta(y_i|y_{i-1}, \ldots, y_1, \mathbf{x})]$$

- This training procedure (teacher forcing) suffers from the problem of *exposure bias*: the model is only ever trained to predict the next token, given a gold standard prefix of a translation of **x**

Neural Machine Translation

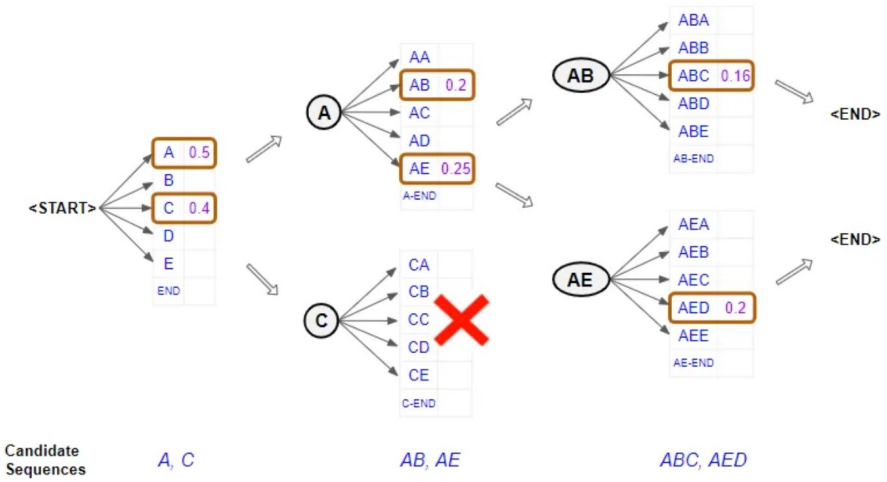
- Decoding:
  - Greedy decoding
    - Iteratively select the most likely token
  - Beam search



Neural Machine Translation: Beam Search

Input: b>0 (beam size), P (probability of next token given prefix)

```
Terminal = []
Q = PriorityQueue()
Q.add(([<START>],1))
while Q not empty:
        a,value=Q.maximum()
        if a ends with <END>:
                Terminal.add(<a,value>)
        else:
                Q.extend([<a.append(v), value*P(v|a)> for v in Vocab])
                Q = maximal b elements in Q
return Terminal.maximum()
```



Exposure Bias

- The exposure bias stems from a discrepancy between training and decoding
  - During training - only gold standard prefixes
  - During decoding - computer generated prefixes

- Possible methods to address this:
  - Train on computer generated prefixes → but how will we know the correct next token then?
  - Use other methods, such as reinforcement learning that bypass the problem
  - Different loss functions